

Projet BookWorld — Rapport final

Sommaire

1. Contexte du projet.....	1
2. Sources de données	1
3. Logique d'extraction	1
4. Requêtes SQL (queries.sql).....	3
5. Nettoyage, enrichissement et agrégation	3
6. Schéma de la base finale.....	4
7. RGPD	5
8. API REST et authentification.....	6
9. Résultats.....	6
10. Limites et améliorations possibles.....	6
11. Conclusion	7

1. Contexte du projet

Ce projet de certification s'appuie sur **BookWorld**, une entreprise fictive de vente de livres en ligne. Elle souhaite mieux piloter son activité à l'international, mais ses données sont aujourd'hui dispersées entre quatre sources hétérogènes : un fichier CSV de ventes brutes, une base SQLite de référentiels métier, un site web (comportant des données fictives) simulant une librairie en ligne à scraper, et une API publique de taux de change. Résultat : toute comparaison de performance entre pays reste longue et sujette à erreur, faute d'une vue consolidée.

L'objectif du projet est de construire un pipeline de données qui centralise ces sources, produit un jeu de données final propre (les ventes agrégées par pays), crée une base SQL exploitable, et expose un indicateur métier via une API REST.

2. Sources de données

Source	Type	Contenu
sales_raw.csv	CSV	Ventes brutes (240) : commande, date, livre, pays, canal, quantité, remise, client...
bookworld_reference.db	SQLite	3 tables. countries (10) : pays, devise, TVA, actif... channels (4) : canal, coût d'acquisition, actif... category_rules (10) : catégorie, marge, canal par défaut, actif...
books.toscrape.com	Scraping (page 1)	Catalogue de livres (20) : prix, catégorie, note, disponibilité...
Frankfurter API (v2)	API REST	Taux de change (1) : GBP → EUR, mis à jour quotidiennement

3. Logique d'extraction

La fonction **extract()** rassemble les 4 sources de données et retourne un dictionnaire de DataFrames pandas (data["sales_raw"], data["category_rules"], data["channels"], data["countries"], data["book_catalog"], data["exchange_rate"]). Ce choix rend le code plus lisible : on accède à chaque source par son nom, plutôt que par sa position dans une liste ou un tuple — plus facile à comprendre et moins source d'erreurs.

Le script est structuré autour d'un point d'entrée clair : le bloc `if __name__ == "__main__":` appelle une fonction `main()` qui enchaîne les trois étapes du pipeline dans l'ordre — `extract()`, puis `transform(data)`, puis `load(results)` — chacune recevant en entrée le résultat de la précédente.

3.1 CSV — `sales_raw.csv`

Lecture directe avec `pandas.read_csv()`, en précisant explicitement l'encodage (`encoding="utf-8"`) pour éviter tout problème sur les caractères accentués des noms de livres ou de clients.

Gestion d'erreur : "tout ou rien". Sans les ventes, aucune étape suivante du pipeline n'a de sens — une `FileNotFoundError` explicite est levée si le fichier est introuvable, et toute autre erreur de lecture (encodage invalide, fichier corrompu) est interceptée, journalisée, puis relancée pour stopper le pipeline proprement.

3.2 SQLite — `bookworld_reference.db`

Trois tables de référence sont lues (`category_rules`, `channels`, `countries`), via une fonction utilitaire `run_query()` qui centralise la connexion et la gestion d'erreur pour éviter de répéter le même bloc `try/except` à chaque requête.

Quatre requêtes supplémentaires sont exécutées à cette étape : trois avec filtre (`is_active = 1`, une par table) et une avec jointure (`category_rules / channels`). Seule la requête filtrée sur `category_rules` et la jointure sont documentées dans `queries.sql` et réutilisées ensuite. Les deux autres requêtes filtrées (`channels`, `countries`) sont exécutées par précaution mais non exploitées en aval – `transform()` refiltrant directement les `DataFrames` complets en `pandas`.

Gestion d'erreur : "tout ou rien". Si la connexion échoue ou qu'une table est illisible, le pipeline s'arrête — les données de référence sont indispensables à l'enrichissement des ventes.

3.3 Scraping — `books.toscrape.com`

Le `scraping` (requests pour les appels HTTP, BeautifulSoup pour le parsing HTML) visite d'abord la première page de la catégorie « All products » (i.e 20 livres), puis la page détail de chaque livre pour en récupérer le prix, la catégorie, la note et la disponibilité.

Sur chaque page détail, la recherche des informations est volontairement limitée à deux zones précises (`product_main` et le tableau "*Product Information*"), plutôt qu'à la page entière. Cette précaution évite un risque concret : plus bas sur la même page, une section "vous aimerez aussi" affiche d'autres livres, avec les mêmes structures HTML — une recherche non limitée pourrait donc, par erreur, récupérer les informations d'un autre livre que celui recherché.

Gestion d'erreur : deux niveaux distincts. La page liste suit la logique "tout ou rien" (sans elle, aucun livre n'est identifiable). La boucle sur les 20 pages détail suit une logique "best effort" : si une page échoue (page indisponible, structure HTML inattendue), le livre concerné est signalé et ignoré, sans bloquer le traitement des 19 autres.

3.4 API — Frankfurter (taux de change)

Un seul appel `GET` vers l'endpoint `v2/rate/GBP/EUR` de l'API Frankfurter (publique, sans clé), qui renvoie directement le taux du jour sous une forme simple (pas de structure imbriquée à parcourir).

Choix de la `v2` plutôt que la `v1`. L'API existe aussi en version `v1` (endpoint `/v1/latest?base=GBP&symbols=EUR`), qui reste supportée indéfiniment et fonctionne tout

aussi bien. La v2 a été préférée car elle propose un endpoint dédié à une seule paire de devises, avec une réponse plus simple à exploiter : la valeur du taux est directement accessible (`data["rate"]`), sans avoir à creuser dans un dictionnaire imbriqué comme en v1 (`data["rates"]["EUR"]`).

Gestion d'erreur : "tout ou rien", avec deux causes distinguées. Une `requests.exceptions.RequestException` signale un problème réseau (service injoignable) ; un (`ValueError`, `KeyError`) signale une réponse reçue mais mal formée (JSON invalide ou clé "rate" absente) — deux messages différents pour deux causes différentes, utiles pour diagnostiquer rapidement le problème.

4. Requêtes SQL (`queries.sql`)

Le fichier `queries.sql` est généré automatiquement par `extract()`, à partir des mêmes variables de requêtes réellement exécutées par le pipeline — garantissant qu'il reste toujours synchronisé avec le code.

Il contient 5 requêtes : 3 extractions simples, une requête avec filtre, et une requête d'enrichissement (jointure) — les deux autres requêtes filtrées exécutées par le pipeline (channels, countries) ne sont pas incluses dans ce fichier, voir section 3.2.

Les 3 extractions simples (SELECT * sur `category_rules`, `channels`, `countries`) ne sont pas reproduites ici ; voici les deux requêtes les plus représentatives :

```
-- Requête avec filtre : catégories actives uniquement
SELECT * FROM category_rules WHERE is_active = 1;

-- Requête d'enrichissement : jointure category_rules / channels
SELECT cr.category_name, cr.default_channel_code,
       cr.is_active AS is_active_cat_rules, cr.margin_rate, cr.strategic_flag,
       c.acquisition_cost_gbp, c.channel_code, c.channel_group, c.channel_name,
       c.is_active AS is_active_channels
FROM category_rules cr
JOIN channels c ON c.channel_code = cr.default_channel_code;
```

5. Nettoyage, enrichissement et agrégation

Cette étape correspond à la fonction `transform()`, qui reprend les données brutes de l'étape précédente pour les nettoyer, les enrichir et les agréger, avant de les transmettre à `load()`. Avant toute jointure, une vérification systématique de la qualité des données (valeurs manquantes, doublons) est effectuée sur chaque source — aucun problème détecté. Les ventes sont ensuite reliées au catalogue scrapé via `book_name`, avec une jointure left : ce choix préserve toutes les ventes, même si un livre venait à manquer du catalogue à cause d'un échec ponctuel du scraping. Une vérification post-jointure confirme qu'aucune vente ne reste orpheline de correspondance (`sales_without_catalog_match = 0`).

Le revenu de chaque ligne de vente est ensuite calculé :

```
revenue_gbp = quantity * price_gbp * (1 - discount_rate)
```

Enfin, ces revenus sont agrégés par pays avec un `groupby("country_code")`, produisant `total_orders`, `total_quantity` et `total_revenue_gbp`, puis convertis en euros grâce au taux de change récupéré via l'API. Le résultat final, `sales_by_country`, contient bien les 6 colonnes minimales attendues : `country_code`, `country_name`, `total_orders`, `total_quantity`, `total_revenue_gbp` et `total_revenue_eur`.

Trois incohérences entre référentiel et ventes réelles ont été identifiées et volontairement conservées, avec avertissement affiché à l'exécution :

- Portugal (PT) et catégorie Horror : marqués `is_active = 0` dans le référentiel, mais des ventes existent bien pour eux.
- Pays-Bas (NL) : ventes existantes mais aucune ligne du tout dans `countries` (ni active, ni inactive) — `country_name` est fixé à "Unknown".
- 7 catégories scrapées (Poetry, Fiction, History...) absentes de `category_rules`, qui ne couvre qu'un sous-ensemble de 10 catégories pour l'exercice.

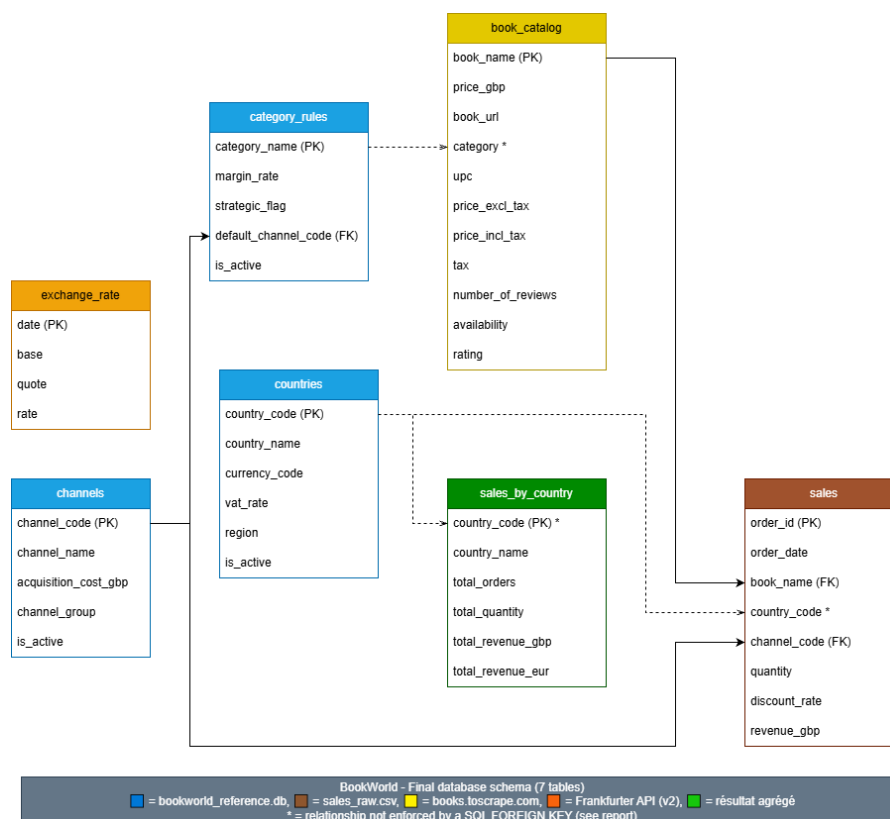
Décision retenue : inclure ces ventes plutôt que les exclure, le référentiel semblant incomplet plutôt que les ventes erronées. Ce choix est documenté explicitement plutôt que masqué.

6. Schéma de la base finale

La base finale **bookworld final.db** comprend 7 tables, reprenant l'ensemble des sources utilisées pour produire **sales_by_country**, plus le résultat agrégé lui-même.

Le diagramme représente les 7 tables de la base finale et leurs relations. Trois tables reprennent telles quelles les données de référence (■ `channels`, `countries`, `category_rules`), une reprend le catalogue scrapé (■ `book_catalog`), une regroupe les ventes nettoyées et conformes RGPD (■ `sales`), une stocke le taux de change utilisé (■ `exchange_rate`, sans relation avec les autres tables), et la dernière contient le résultat agrégé final (■ `sales_by_country`).

Les clés étrangères strictes (`sales.book_name` → `book_catalog.book_name`, `sales.channel_code` → `channels.channel_code`, `category_rules.default_channel_code` → `channels.channel_code`) garantissent l'intégrité référentielle partout où le référentiel est complet.



Les relations marquées d'un astérisque (*) sur le diagramme – sales.country_code, sales_by_country.country_code, book_catalog.category – ne sont volontairement pas déclarées comme FOREIGN KEY dans schema_final.sql : une contrainte stricte rejeterait l'insertion des cas documentés en section 5 (NL, catégories hors référentiel). Les autres relations (book_name, channel_code, default_channel_code) restent des clés étrangères strictes, vérifiées à l'insertion.

Chargement : append plutôt que replace, avec suppression préalable du fichier.

load() écrit chaque table avec to_sql(..., if_exists="append") plutôt que "replace" : "replace" supprimerait la table et laisserait pandas la recréer avec son propre schéma déduit automatiquement, perdant les contraintes PRIMARY KEY/FOREIGN KEY définies dans schema_final.sql. "append" seul insérerait dans la table déjà existante, mais dupliquerait les lignes si le pipeline est exécuté plusieurs fois sans vider la base au préalable.

Pour obtenir à la fois les contraintes du schéma et l'absence de doublons entre exécutions, load() supprime output_pipeline/bookworld_final.db (s'il existe) avant de recréer le schéma depuis zéro : chaque exécution repart d'une base propre et correctement contrainte. Vérifié : deux appels successifs à load() avec les mêmes données produisent 1 seule ligne (pas 2) dans chaque table.

7. RGPD

Le **RGPD (Règlement Général sur la Protection des Données)** est une réglementation européenne, en vigueur depuis 2018, qui encadre la collecte, le stockage et l'utilisation des données personnelles des individus par les entreprises et organisations. Une donnée personnelle est toute information permettant d'identifier une personne physique, directement (nom, prénom, email) ou indirectement (combinaison d'informations). Son objectif est de protéger la vie privée des personnes dont les données sont traitées, en encadrant strictement ce que les entreprises ont le droit de faire de ces informations.

Un des principes centraux du RGPD est la **minimisation des données** : une organisation ne doit collecter et conserver que les données strictement nécessaires à l'usage prévu. Si une donnée personnelle n'est pas indispensable à l'objectif poursuivi, elle ne doit pas être conservée, même si elle est présente dans les données sources d'origine.

Application à BookWorld :

sales_raw.csv contient les prénoms et noms des clients (customer_first_name, customer_last_name) – des identifiants directs, donc des données personnelles au sens du RGPD. Or, l'usage final de la base construite dans ce projet est de produire un indicateur agrégé (les ventes par pays) : à aucun moment il n'est nécessaire de savoir qui, individuellement, a acheté quoi. Conserver ces colonnes dans la base finale n'apporterait donc aucune valeur à l'usage prévu, tout en exposant inutilement des données personnelles.

Par principe de minimisation, ces colonnes sont exclues au niveau du schéma lui-même (schema_final.sql), pas seulement filtrées après coup dans les requêtes :

```
CREATE TABLE IF NOT EXISTS sales (
    order_id TEXT PRIMARY KEY, order_date TEXT, book_name TEXT,
    country_code TEXT, channel_code TEXT, quantity INTEGER,
    discount_rate REAL, revenue_gbp REAL,
    FOREIGN KEY (book_name) REFERENCES book_catalog (book_name),
    FOREIGN KEY (channel_code) REFERENCES channels (channel_code)
);
```

Ce choix est plus robuste qu'un simple filtrage de colonnes après coup : même une erreur future dans load() ne pourrait pas réintroduire ces données par accident, la structure de la

table ne les prévoyant simplement pas. C'est une application concrète du principe de "**privacy by design**" (protection de la vie privée dès la conception), qui découle directement de la minimisation des données : plutôt que de restreindre l'accès à une information sensible après l'avoir collectée, on évite de la faire transiter jusqu'à la base finale. **Les données de ce projet étant fictives, le RGPD ne s'applique pas légalement à proprement parler** ; ce traitement illustre une bonne pratique transposable à un contexte réel.

8. API REST et authentification

Méthode	Route	Auth	Description
GET	/health	Non	{"status": "ok"} si l'API fonctionne
GET	/sales-by-country	Oui (token)	Données agrégées en JSON

La route /sales-by-country est protégée par un token simple, comparé à la valeur de la variable d'environnement BOOKWORLD_API_TOKEN. Cette variable est définie dans un fichier .env local, jamais commité sur GitHub ; un fichier .env.example sert de modèle pour indiquer le format attendu, sans exposer de vraie valeur secrète.

Le client transmet son token via l'**en-tête HTTP Authorization: Bearer <token>** — la pratique standard pour une API REST en production, car le token ne transite jamais dans l'URL : il n'apparaît donc ni dans les journaux du serveur, ni dans l'historique du navigateur. Une première version du projet acceptait aussi le token en paramètre d'URL (?token=...), par commodité pour tester la route directement dans un navigateur. Ce choix a été retiré puisque l'en-tête reste la seule pratique réellement recommandée pour un déploiement en production.

En cas d'erreur, la route ne renvoie jamais un plantage Flask brut : elle répond 503 si la base finale n'existe pas encore (le pipeline n'a pas encore été exécuté), et 500 si une table attendue est illisible.

L'API est lancée avec `app.run(debug=True)`, qui démarre le serveur de développement intégré de Flask (Werkzeug) — pratique pour développer (rechargement automatique, messages d'erreur détaillés), **mais non conçu pour la production** : il ne gère pas la charge, et le mode debug expose la stack trace complète en cas d'erreur. Un déploiement réel nécessiterait un serveur de production et la désactivation du mode debug.

9. Résultats

Après une exécution complète du pipeline, `sales_by_country` agrège les 240 ventes par pays (taux de change GBP→EUR du 13/07/2026, sujet à variation d'une exécution à l'autre). Top 5 des pays par revenu :

Pays	Nom	Commandes	Revenu (£)	Revenu (€)
DE	Germany	40	4 014.70	4 654.65
FR	France	56	3 979.20	4 613.49
BE	Belgium	30	2 682.30	3 109.86
ES	Spain	28	1 992.23	2 309.79
PT	Portugal	20	1 985.03	2 301.44

- Revenu total : environ **20 059 £ (~23 256 €)** sur 240 commandes tous pays confondus ; l'Allemagne et la France génèrent à elles seules plus de **39 % du total**.
- Le canal WEB domine (**12 680 £, ~63 %**), loin devant APP (4 375 £) et MKT (3 004 £) ; les pays hors référentiel actif (Portugal, Pays-Bas – voir section 5) génèrent eux aussi un revenu non négligeable.

10. Limites et améliorations possibles

- **Données personnelles dans le fichier source versionné** : sales_raw.csv (noms/prénoms clients en clair) reste commité tel quel sur Git. La minimisation RGPD protège la base finale (section 7), pas ce fichier — à anonymiser ou sortir du contrôle de version pour une mise en production.
- **Peu de nettoyage nécessaire** : sales_raw.csv et bookworld_reference.db ne contenaient ni valeur manquante ni doublon (section 5). Des données réelles seraient généralement plus "sales" (formats incohérents, valeurs aberrantes) ; les vérifications systématiques restent utiles pour détecter ce cas, mais l'exercice n'a pas permis de démontrer un nettoyage plus poussé.
- **Intégrité référentielle partielle** : trois relations (country_code ×2, category) ne sont pas garanties par une contrainte SQL, faute d'un référentiel complet. En production, il faudrait aligner le référentiel ou gérer explicitement les "pays/catégories inconnus".
- **Scraping séquentiel** : les 20 pages détail sont visitées une par une, sans parallélisation ni délai — acceptable ici, à revoir pour un catalogue plus large.
- **Authentification simple** : un token unique partagé, sans expiration ni gestion multi-utilisateurs — suffisant pour l'exercice, mais à renforcer pour un usage réel.
- **API accessible uniquement en local** : http://127.0.0.1:5000 n'est joignable que depuis la machine hôte. Pour une démo par un tiers, un tunnel ngrok (ngrok http 5000) peut l'exposer temporairement — pratique ponctuellement, pas pour de la production (URL changeante, machine devant rester allumée).

11. Conclusion

Ce projet a permis de mettre en œuvre l'ensemble du cycle de vie d'une donnée : extraction depuis [4 sources hétérogènes](#) (CSV, SQLite, scraping web, API REST), nettoyage et détection systématique des problèmes de qualité, enrichissement et agrégation avec pandas, modélisation d'un schéma relationnel contraint par des clés primaires et étrangères, et exposition du résultat via une API REST sécurisée par token.

Au-delà de l'aspect technique, ce travail illustre des compétences propres à l'analyse de données : [documenter](#) et [justifier](#) chaque choix (gestion d'erreur différenciée selon la source, jointures, contraintes SQL), plutôt que masquer les incohérences entre référentiel et données réelles, les rendre visibles et argumenter la décision de les conserver. Les pistes d'amélioration identifiées (référentiel à compléter, authentification à renforcer, anonymisation des données sources) montrent une conscience claire des limites d'un projet réalisé dans un cadre pédagogique, et des étapes nécessaires pour un passage en production.